

Edge of Arcane Command

～非同期処理×拡張性で作る“動くコマンドバトル”～



制作者名：田原陽紀

目次

基本情報

コンセプト

- ▶ ゲーム性の差別化ポイント
- ▶ 技術コンセプト

ゲーム概要

- ▶ バトルの流れ・操作解説

実装詳細

- ▶ 非同期処理（UniTaskとCoroutine）
- ▶ ステータス管理（継承+Json+SO）

技術工夫と苦労

- ▶ 工夫した点
- ▶ 苦労した点と解決策

今後の展望

基本情報

- ▶ タイトル : **Edge of Arcane Command**
- ▶ ゲームジャンル : ターン性コマンドバトル
- ▶ 制作人数 : 1 人 (個人開発)
- ▶ 制作者名 : 田原陽紀
- ▶ 使用エンジン : Unity
- ▶ お借りしたツール : UniTask、newtonsoft.json



↑ Unity C# 実装

ゲームコンセプト

～プレイヤーが戦略を練りつつ没入できる、リソース型コマンドバトル～

ゲームの差別化ポイント

- ▶ リソース管理型コマンドバトル×状況依存ギミック
- ▶ キャラ間のシナジーを考慮したターン進行

技術コンセプト

- ▶ 保守性 / 拡張性重視
- ▶ UniTaskとCoroutineによる非同期制御で「止まらない没入感」
- ▶ 継承+SO（スクリプタブジェクト）、Jsonでステータス管理を汎用化

バトルの流れ

バトルはターン性コマンドバトル

プレイヤーターン ✂

- ▶ 左から順番に、各キャラクターが行動を選択
- ▶ スキル・必殺技・回復・バフを使い分けて敵を攻略
- ▶ 攻撃時は対象を自分で選択可能

エネミーターン 💀

- ▶ 敵はランダムに行動し、強力な全体攻撃や状態異常を使う場合もある



ゲームについて操作方法

キー割り当て



Aキー：通常攻撃



Sキー：スキル

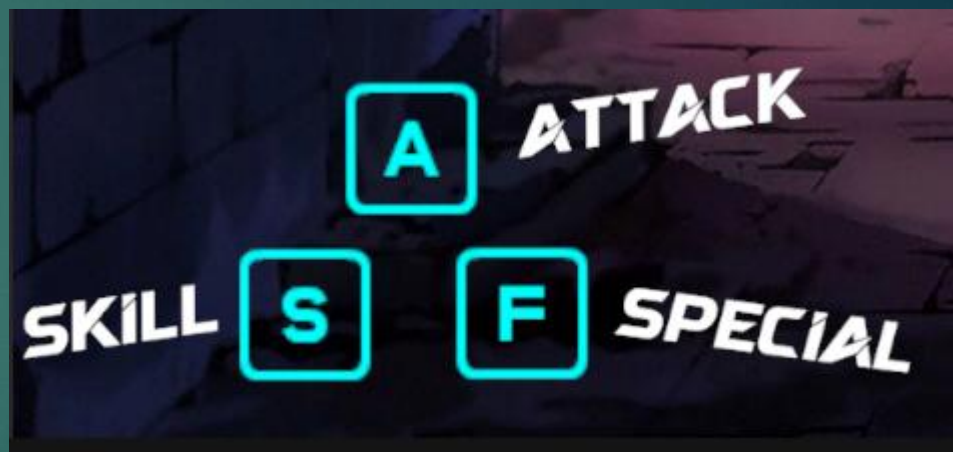


Fキー：必殺発動

ESC：メニュー中断



右下のボタン：プレイヤー状態確認画面を開く



プレイヤーキャラ紹介：スキル・必殺技 紹介

アタッカー

近接攻撃を得意とした剣士

攻撃力が高いのが特徴

ウィザードスキル：2段切り
攻撃力が2倍になり敵に攻撃する

スペシャル：敵全体に攻撃する



バッファー

魔法の力で戦う魔女

攻撃力は低めだが味方の強化を得意としている

ウィザードスキル：エンハンス
味方の攻撃力が2倍になる

スペシャル：味方全員強化する



ヒーラー

魔法（妖術）を使って
味方の回復を得意としている
支援キャラ

ウィザードスキル：ヒーリング
味方単体を回復する

スペシャル：味方全員を回復する



敵キャラ紹介

スライム（初級エネミー）

- ▶ HP：低
- ▶ 攻撃低め
- ▶ 行動：通常攻撃のみ
- ▶ 戦略：序盤で数を裁きやすいので全体攻撃で一掃すると楽



スケルトン（デバフエネミー）

- ▶ HP：中
- ▶ 能力：確率で攻撃力をダウン
- ▶ 連携：デバフ中に確率で追加デバフを付与
- ▶ 戦略：まずバフで自分の攻撃力を高め、安全に攻撃



敵キャラ紹介

ドラゴン（ラスボスエネミー）

- ▶ HP：高
- ▶ 攻撃力：高（複数パターンの攻撃を使い分け）
- ▶ 特殊能力：必殺を受けると「やけど」状態 → 毎ターンダメージ（2ターン継続）
- ▶ 戦略：
 - ▶ 高火力をいかに耐えられるかが鍵
 - ▶ 回復をタイミングよく挟むことが攻略のポイント



プログラムについて

挑戦（チャレンジ）

- ▶ **UniTask + Coroutine で非同期制御を導入**

↳ バトル中の処理を非同期化

- ▶ **継承 + ScriptableObject + JSON**

↳ ステータスの管理を汎用化

成果

- ▶ **中断のないシームレスなゲーム体験に成功**

- ▶ **開発効率の向上！**

↳ 新キャラクターの追加が継承とSOだけで完結

バトルについて

▶ ターン性コマンドバトル：戦略性×没入感

↳ 結果：UniTask + Coroutineの非同期処理で中断のないシームレスなゲーム体験を提供

▶ UI表示とエフェクトの表示時間をフレームや秒数で管理し、一時停止などを非同期で行っています。



```
//UIマネージャーからプレイヤーターン表示
UIManager.Instance.PlayerTurnUI.SetActive(true);

//1フレーム待つ（キャンセルトークンが呼ばれたらキャンセル）
await UniTask.Delay(TimeSpan.FromSeconds(TurnDelay), cancellationToken: token);

//全プレイヤーの行動フラグをリセット
attacker.ResetActionFlag();
buffer.ResetActionFlag();
healer.ResetActionFlag();

//UIマネージャーからプレイヤーターン非表示
UIManager.Instance.PlayerTurnUI.SetActive(false);

//UIマネージャーからプレイヤーターン時に表示するUIを表示
UIManager.Instance.StartPlayerTurnUI();
```

プレイヤーターンUIの表示をUniTaskを用いて数秒後にUIが非表示になるよう実装しています

キャラクターステータス：再利用×拡張性

継承

▶ 再利用性向上

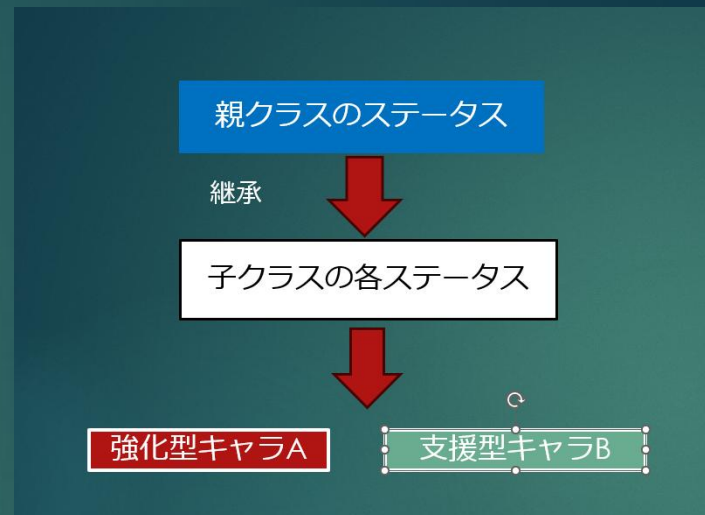
└ 共通ロジックを親クラスで一元化

▶ 拡張も簡単

└ 新たな役職（○○クラス）を子クラス追加するだけ

▶ 細かなキャラ差別化

└ 支援A／強化Bなど、特定役割向けクラスを複数定義可能



Scriptable Objectによるパラメータ管理の汎用化

～アセットの汎用化×外部連携でメンテナンス負荷80%削減を実現～



外部連携



Asset化

- ▶ データ仕様をAsset化（IDだけでロード）
- ▶ メンテナンス負荷を80%削減（仕様変更が容易に）
- ▶ 外部テキストデータ連携対応（Json/CSVで拡張自在）
- ▶ 新技術の習得（ScriptableObject設計を実践）

Scriptable Objectによるパラメータ管理コード

```
using System;
using UnityEngine;

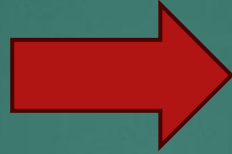
/// <summary>
/// プレイヤーのデータベースで利用するパラメータ
/// </summary>
[Serializable]
public class PlayerParameters
{
    [Tooltip("プレイヤーの名前データ")]
    public string PlayerNameData;

    [Tooltip("プレイヤーの名前の最大体力のデータ")]
    public int PlayerMaxHPData;

    [Tooltip("プレイヤー攻撃力データ")]
    public int PlayerAttackPowerData;

    [Tooltip("プレイヤーのバフパワーデータ")]
    public int BuffPowerData;

    [Tooltip("プレイヤーの回復力データ")]
    public int HealPowerData;
}
```



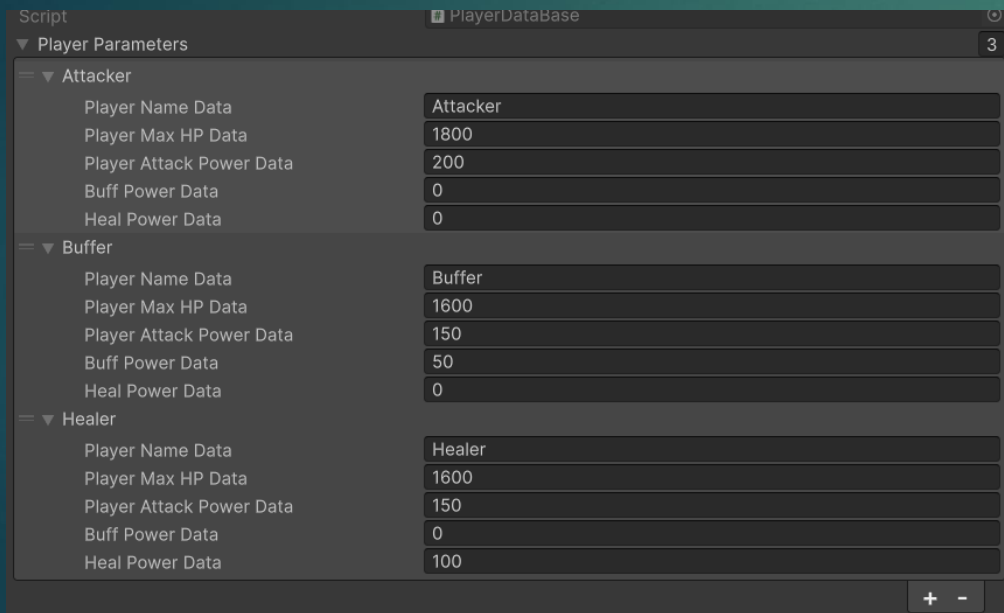
```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// プレイヤーのスクリタブルオブジェクトを作成
/// </summary>
[CreateAssetMenu(fileName = "PlayerDataBase", menuName = "ScriptableObject/PlayerDataBase")]
public class PlayerDataBase : ScriptableObject
{
    //プレイヤーのパラメータのリスト
    public List<PlayerParameters> PlayerParameters = new List<PlayerParameters>();
}
```

必要なパラメータを作成し、リスト化

Scriptable Objectによるパラメータ管理コード

パラメータを一括管理
Unity内で編集可能



実行時にSOからパラメータをロード

```
/// <summary>  
/// パラメータを設定するメソッド  
/// プレイヤーのデータベースから読み込み  
/// </summary>  
/// 2 個の参照  
protected override void SetPlayerParameters()  
{  
    // LINQを使い、プレイヤーデータベースからPlayerIDと一致するプレイヤー情報を取得  
    var playerData = PlayerDataBase.PlayerParameters.FirstOrDefault(p => p.PlayerNameData == PlayerID);  
  
    // 一致するプレイヤー情報が見つかった場合、パラメータを設定  
    if (playerData != null)  
    {  
        //最大体力のデータを読み込み  
        PlayerMaxHP = playerData.PlayerMaxHPData;  
  
        //アタッカーの現在のHPを最大に設定してHPバーも最大に設定  
        PlayerCurrentHP = PlayerMaxHP;  
  
        Debug.Log($"PlayerCurrentHP: {PlayerCurrentHP}, PlayerMaxHP: {PlayerMaxHP}");  
  
        PlayerHPBar.maxValue = PlayerCurrentHP;  
        PlayerHPBar.value = PlayerCurrentHP;  
        PlayerHPBar.minValue = 0;  
  
        //攻撃力をアタッカーのデータの攻撃力を読み込む  
        AttackPower = playerData.PlayerAttackPowerData;  
  
        //初期攻撃力もアタッカーの攻撃力に設定  
        PlayerResetAttackPower = AttackPower;  
  
        //生存状態にする  
        IsAlive = true;  
    }  
}
```

JSONによるテキスト外部化

～ID呼び出しで動的ロード～

- ▶ テキストをJSON外部管理
 - ↳ ID呼び出しで全文言ロード
- ▶ 編集作業を50%削減
 - ↳ JSON書換→即反映

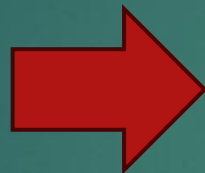


Jsonによるテキストデータ管理

チュートリアルやバトル時のテキストをJsonで管理

```
{
  "id": "BufferOnDebuff",
  "text": "バッファーにデバフが付与された!"
},
{
  "id": "HealerOnDebuff",
  "text": "ヒーラーにデバフが付与された!"
}
```

JSONのデータ
(シナリオのスクリプトのように作成)



```
//アタッカーのターン開始通知表示
BattleActionTextManager.Instance.ShowBattleActionText("AttackerTurnText");
```

```
33 個の参照
public void ShowBattleActionText(string id)
{
    //JSONから読み込んだデータをいれるリストからidを取得
    BattleText battleText = battleTextConvertedList.Find(text => text.id == id);

    if (battleText != null)
    {
        //IDをもとにテキストを表示しテキストウィンドウも表示
        characterBattleActionText.text = battleText.text;
        textWindow.SetActive(true);
    }
    else
    {
        Debug.LogWarning($"指定された ID ({id}) に対応するテキストが見つかりません");
    }
}
```

状況に応じてIDを呼び出しロード



セーブ&ロード機能の実装

～初回はSO、以降はJSONでデータ永続化～

- ▶ 初回起動時：ScriptableObjectによる初期データ読み込み

└プレイヤーの初期レベル／ステータスをSOからロード

- ▶ それ以降：JSONファイルからセーブデータ復元

└セーブデータが存在すればJSONを読み込み、全パラメータを復元

レベルアップ時：JSONへの書き出し処理

└レベル／経験値の更新を即時JSONに保存

拡張性とメンテナンス性を両立

└SO→JSONという遷移をフロー化し、

└SOで初期値管理 → JSONでランタイム永続化」の仕組みを確立

プレイヤーデータを保存するコード内容

プレイヤーデータを保存するコード

Jsonのプレイヤーデータ

```
/// <summary>
/// Jsonにレベルアップしたランタイムのパラメータデータを保存するメソッド
/// </summary>
/// <param name="players">レベルアップするプレイヤー</param>
/// 1 個の参照
public static void SavePlayers(List<BasePlayerStatus> players)
{
    //レベルアップするプレイヤーのリストに入っているキャラのランタイムデータを保存
    foreach (var player in players)
    {
        var data = new PlayerSaveData
        {
            playerID_SaveData = player.PlayerID,
            level_SaveData = player.Level,
            attackPower_SaveData = player.AttackPower,
            playerMaxHP_SaveData = player.PlayerMaxHP,
        };

        //Jsonの文字列に変換して保存（出力Jsonをインデントする）
        string json = JsonConvert.SerializeObject(data, Formatting.Indented);

        //保存パス、ファイル名指定
        string path = Application.persistentDataPath + $"/{player.PlayerID}_save.json";

        //指定した保存パスにJsonを書き込み
        File.WriteAllText(path, json);
    }
}
```

A screenshot of a code editor window showing a file named 'Attacker_save.json'. The file contains a JSON object with the following data: playerID_SaveData: "Attacker", level_SaveData: 1, attackPower_SaveData: 3100, and playerMaxHP_SaveData: 2300. The editor has a dark theme and a sidebar on the left with icons for file operations.

```
{
  "playerID_SaveData": "Attacker",
  "level_SaveData": 1,
  "attackPower_SaveData": 3100,
  "playerMaxHP_SaveData": 2300
}
```

ステージクリア時にプレイヤーのレベルを上げ、そのデータが自分のデバイスに保存されます。

プレイヤーデータをロードする内容

初回時（セーブ無し）はSOからロード

セーブデータがあればセーブデータから読み込み

```
/// <summary>
/// パラメータを設定するメソッド
/// プレイヤーのデータベースから読み込み
/// </summary>
2 個の参照
protected override void SetPlayerParameters()
{
    // LINQを使い、プレイヤーデータベースからPlayerIDと一致するプレイヤー情報を取得
    var playerData = PlayerDataBase.PlayerParameters.FirstOrDefault(p => p.PlayerNameData == PlayerID);

    // 一致するプレイヤー情報が見つかった場合、パラメータを設定
    if (playerData != null)
    {
        // 最大体力のデータを読み込み
        PlayerMaxHP = playerData.PlayerMaxHPData;

        // アタッカーの現在のHPを最大に設定してHPバーも最大に設定
        PlayerCurrentHP = PlayerMaxHP;

        Debug.Log($"PlayerCurrentHP: {PlayerCurrentHP}, PlayerMaxHP: {PlayerMaxHP}");

        PlayerHPBar.maxValue = PlayerCurrentHP;
        PlayerHPBar.value = PlayerCurrentHP;
        PlayerHPBar.minValue = 0;

        // 攻撃力をアタッカーのデータの攻撃力を読み込む
        AttackPower = playerData.PlayerAttackPowerData;

        // 初期攻撃力もアタッカーの攻撃力に設定
        PlayerResetAttackPower = AttackPower;

        // 生存状態にする
        IsAlive = true;
    }
}
```

```
/// <summary>
/// セーブデータがあればセーブしたプレイヤーデータからステータスを読み込む
/// </summary>
6 Unity メッセージ 16 個の参照
protected virtual void Start()
{
    // プレイヤーIDのセーブデータを取得
    string path = Application.persistentDataPath + $"{PlayerID}_save.Json";

    // セーブデータがあれば読み、0#に変換した後、ステータスロード処理開始
    if (File.Exists(path))
    {
        string json = File.ReadAllText(path);
        PlayerSaveData saveData = JsonConvert.DeserializeObject<PlayerSaveData>(json);

        // セーブデータからステータスを読み込む
        ApplySaveData(saveData);
    }

    // セーブデータがなければスクリプタブルオブジェクトからロード
    else
    {
        SetPlayerParameters();
    }
}
```

アピールポイントまとめ：工夫した点

非同期処理で「止まらない没入感」を実現

- ▶ UniTask + Coroutineを組み合わせ、ターン中のステージ演出や全体攻撃の演出をフレーム単位で途切れさせず同期。
- ▶ 複数演出の連携やキャンセル処理を安定化させ、プレイヤー視点で“止まらない操作感”を達成。

新技術への積極的チャレンジ

- ▶ ScriptableObject (SO) / JSONによるデータ管理を導入し、パラメータ追加やテキスト編集をプログラミング不要で行えるように設計。
- ▶ 初回起動時はSO、以降はJSONからロードする「SO→JSON移行フロー」を確立し、将来的な仕様変更やコンテンツ拡張を容易にした。

苦勞して得た成長

1. UniTaskのキャンセル処理の実装

課題：どのタイミング・どの場所で処理をキャンセルすべきか判断が難しかった

対応：サンプルコードやドキュメントを参照しつつ、試行錯誤の末、正常動作を実現

2. 敵選択UIの不具合修正

課題：クリックした敵にUIが正しく追従せず、原因特定に時間を要した

対応：原因を精査し、クリック時にUIを移動する方式に変更して解消

今後の目標

今回の制作では、新しいことに挑戦し、バグや問題に対処しながら解決策を見出す力を養うことができました。

今後、さらなる成長を目指すためには...

【技術面の目標】

- ▶ 今回は2Dでの作成だったが3Dゲームの制作にも挑戦し、より技術力を高めたい

【作品の拡張計画】

- ▶ 今回のプロジェクトで得た知識を活かし、新キャラクターや新しいバトルギミックを追加したステージを作成し、ゲーム体験の幅を広げたい

これらを実現することで、より多くのプレイヤーに面白い作品を目指しながら、継続的に成長していきたいと考えています。